

CPS 3440/5014: Analysis of Algorithms

Dr. Yan Ma

yama@kean.edu

yan.ma@kean.edu

Insertion Sort (Review)

- An algorithm for the sorting problem
- Intuition:

https://www.youtube.com/watch?app=desktop&v=Q1JdRUh1_98

- Algorithm visualization:

<https://www.youtube.com/watch?v=IniSptbttgg>

Insertion Sort

What is the precondition for this loop?

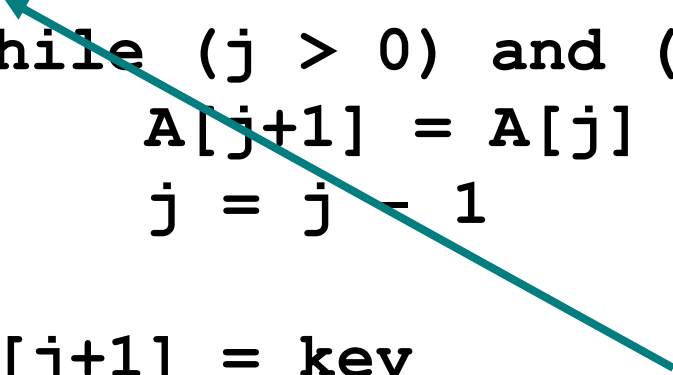


```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

Insertion Sort

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

How many times will this loop execute?



Insertion Sort

Statement	Effort
InsertionSort(A, n) {	
for i = 2 to n {	c_1n
key = A[i]	$c_2(n-1)$
j = i - 1;	$c_3(n-1)$
while (j > 0) and (A[j] > key) {	c_4T
A[j+1] = A[j]	$c_5(T-(n-1))$
j = j - 1	$c_6(T-(n-1))$
}	0
A[j+1] = key	$c_7(n-1)$
}	0
}	

$T = t_2 + t_3 + \dots + t_n$ where t_i is number of while expression evaluations for the i^{th} for loop iteration.

Analyzing Insertion Sort

- $T(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4T + c_5(T - (n-1)) + c_6(T - (n-1)) + c_7(n-1)$
 $= c_8T + c_9n + c_{10}$
- What can T be?
 - Best case -- inner loop body never executed
 - $t_i = 1 \rightarrow T(n)$ is a linear function
 - Worst case -- inner loop body executed for all previous elements
 - $t_i = i \rightarrow T(n)$ is a quadratic function
 - Average case
 - ???

Analysis

- Simplifications
 - Ignore actual and abstract statement costs
 - *Order of growth* is the interesting measure:
 - Highest-order term is what counts
 - Remember, we are doing asymptotic analysis
 - As the input size grows larger it is the high order term that dominates

Upper Bound Notation

- We say InsertionSort's run time is $O(n^2)$
 - Properly we should say run time is *in* $O(n^2)$
 - Read O as “Big-O” (you’ll also hear it as “order”)
- In general, a function
 - $f(n)$ is $O(g(n))$ if there exist **positive constants** c and n_0 such that $f(n) \leq c \cdot g(n), \forall n \geq n_0$
- Formally
 - $O(g(n)) = \{ f(n): \exists \text{ positive constants } c \text{ and } n_0 \text{ such that } f(n) \leq c \cdot g(n) \forall n \geq n_0 \}$

Insertion Sort Is $O(n^2)$

- Proof
 - Suppose runtime is $an^2 + bn + c$
 - If any of a , b , and c are less than 0 replace the constant with its absolute value
 - $an^2 + bn + c \leq (a + b + c)n^2 + (a + b + c)n + (a + b + c)$
 - $\leq 3(a + b + c)n^2$ for $n \geq 1$
 - Let $c' = 3(a + b + c)$ and let $n_0 = 1$

Insertion Sort Is $O(n^2)$

- Question
 - Is InsertionSort $O(n^3)$?
 - Is InsertionSort $O(n)$?

Big O Fact

- A polynomial of degree k is $O(n^k)$
- Proof:
 - Suppose $f(n) = b_k n^k + b_{k-1} n^{k-1} + \dots + b_1 n + b_0$
 - Let $a_i = |b_i|$
 - $f(n) \leq a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$
$$\leq n^k \sum a_i \frac{n^i}{n^k} \leq n^k \sum a_i \leq cn^k$$

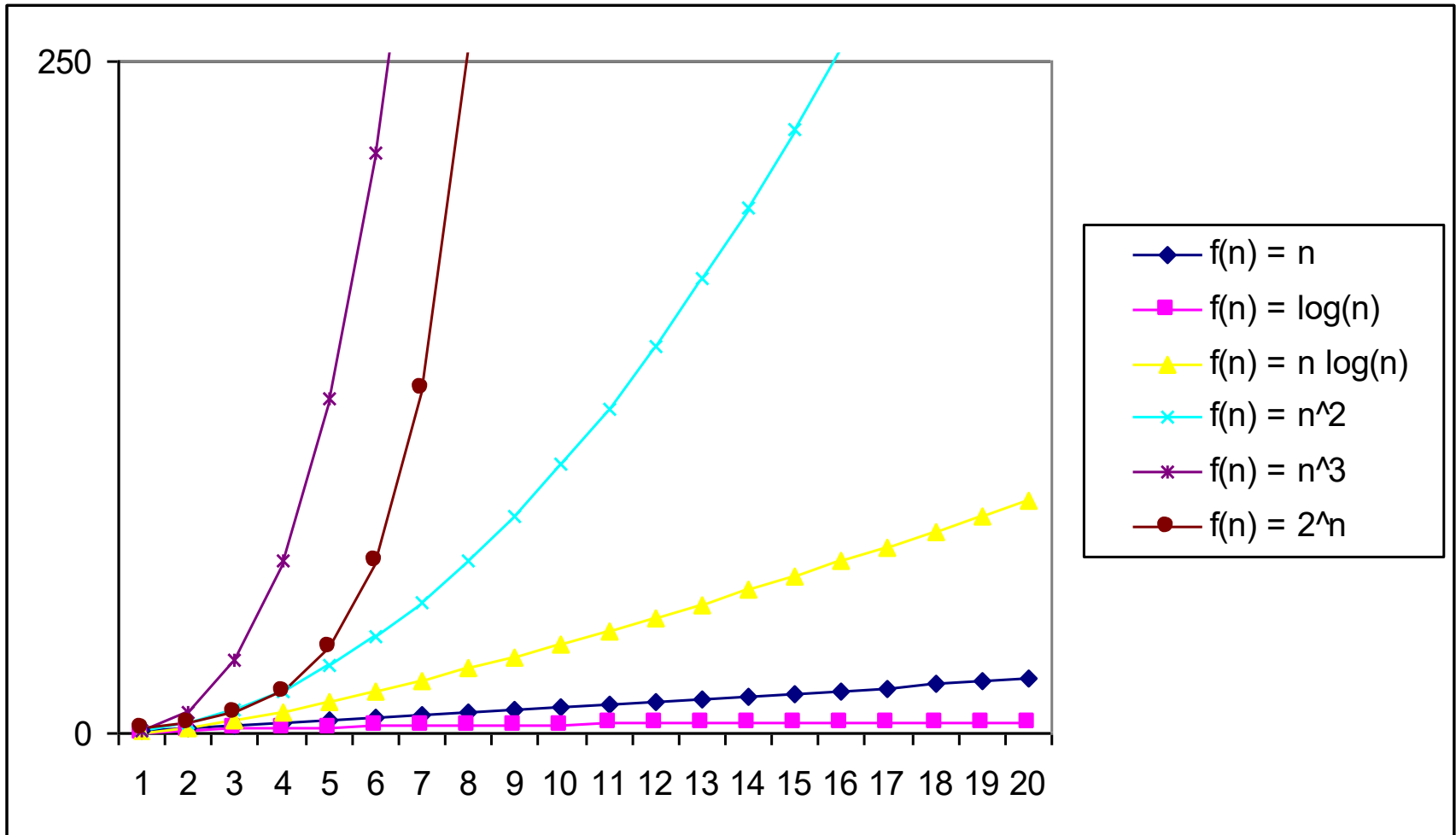
Lower Bound Notation

- We say InsertionSort's run time is $\Omega(n)$
- In general, a function
 - $f(n)$ is $\Omega(g(n))$ if $\exists$ positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n), \forall n \geq n_0$
- Proof:
 - Suppose run time is $a \cdot n + b$
 - Assume a and b are positive (what if b is negative?)
 - $a \cdot n \leq a \cdot n + b$

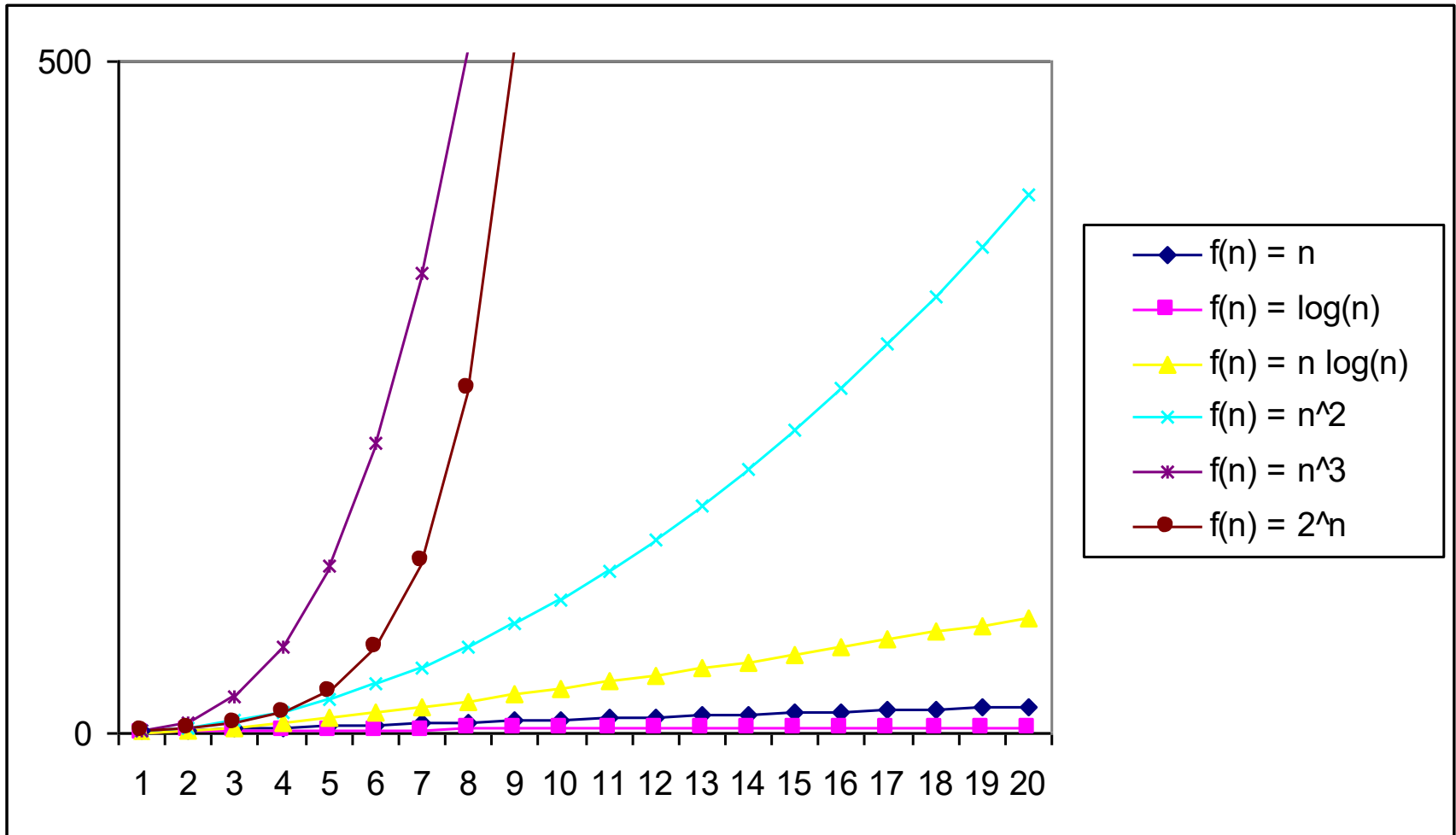
Asymptotic Tight Bound

- A function $f(n)$ is $\Theta(g(n))$ if $\exists$ positive constants c_1 , c_2 , and n_0 such that
$$c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0$$
- Theorem
 - $f(n)$ is $\Theta(g(n))$ *iff* $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$
 - Proof: someday

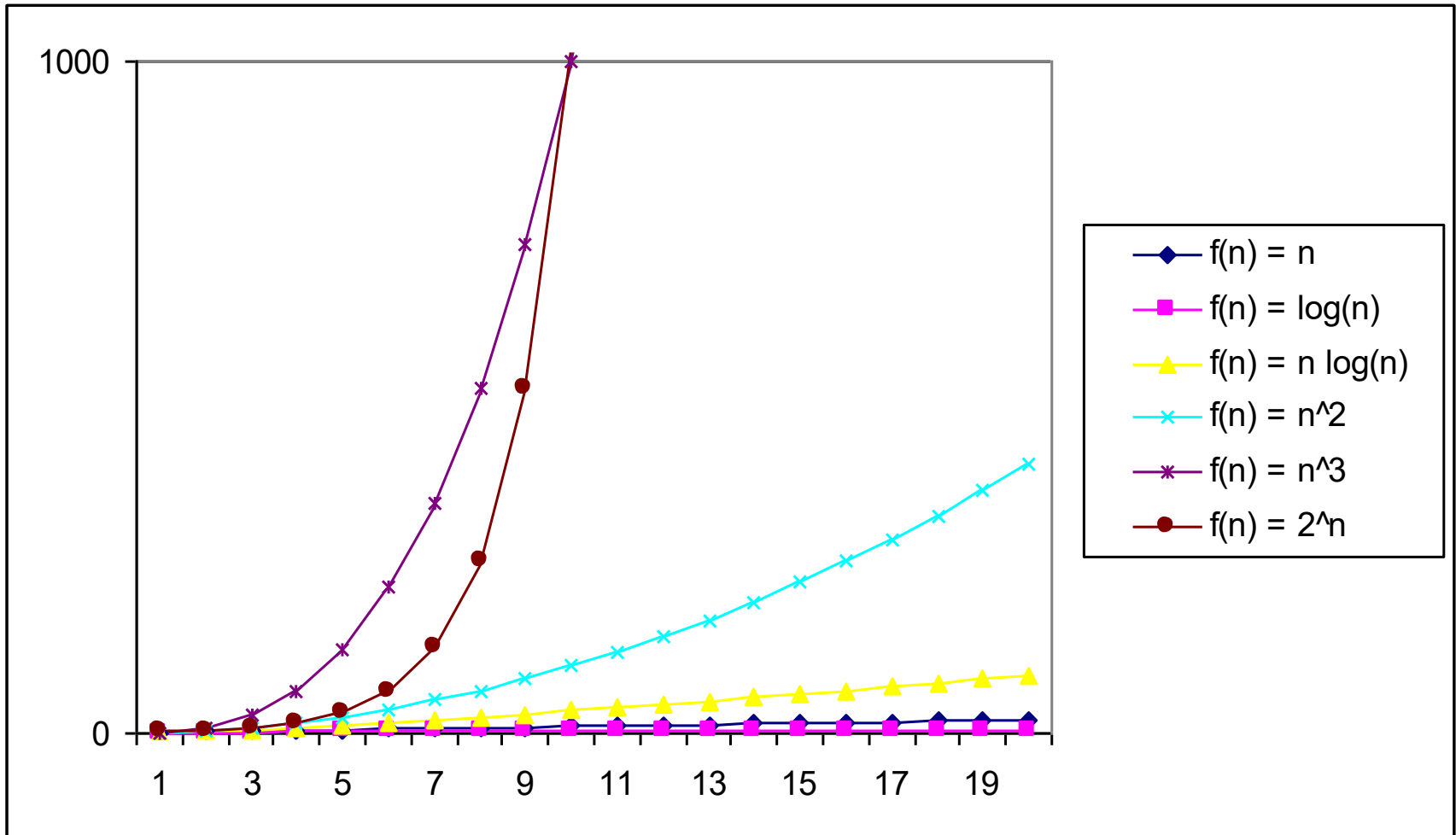
Practical Complexity



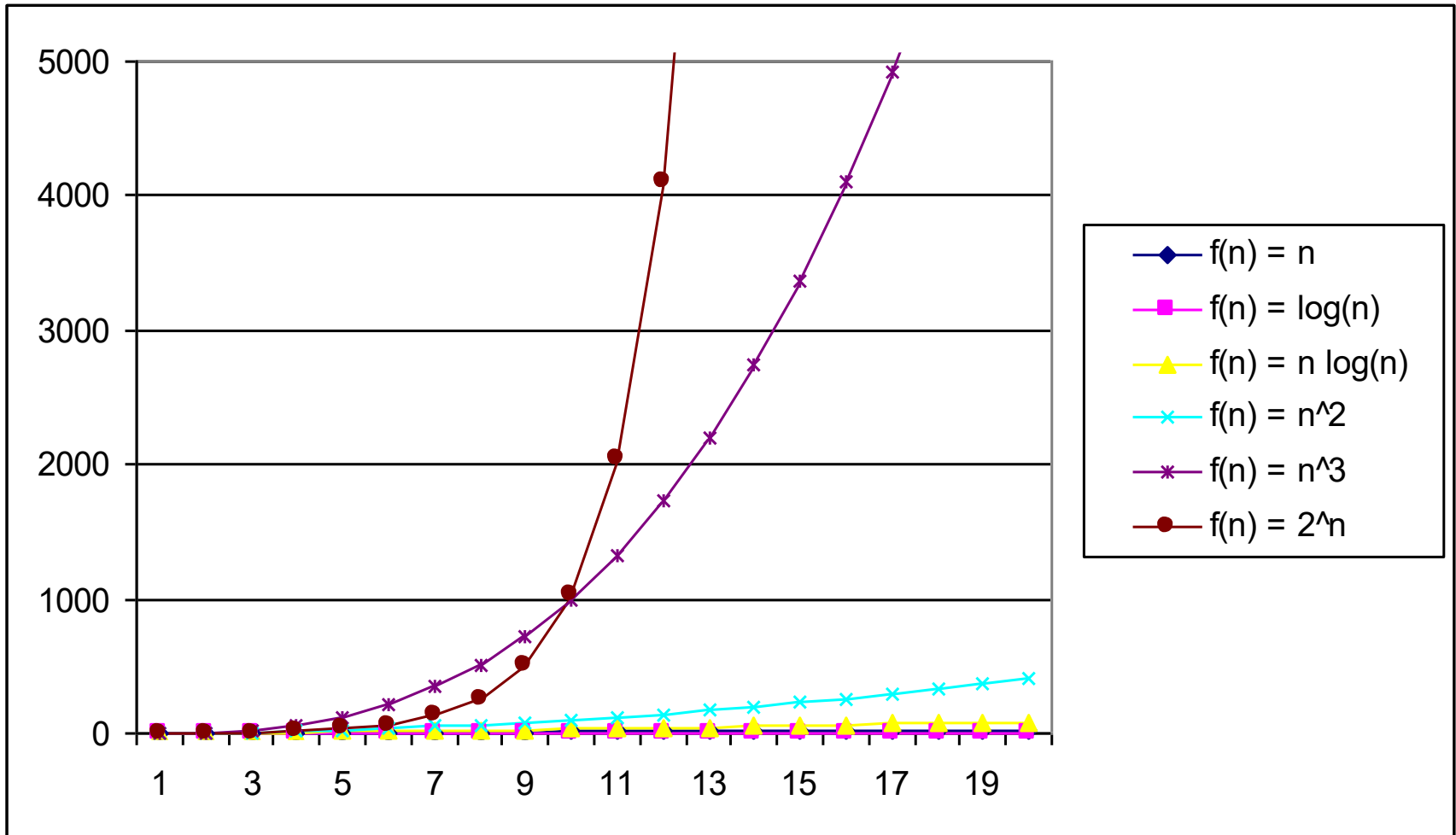
Practical Complexity



Practical Complexity

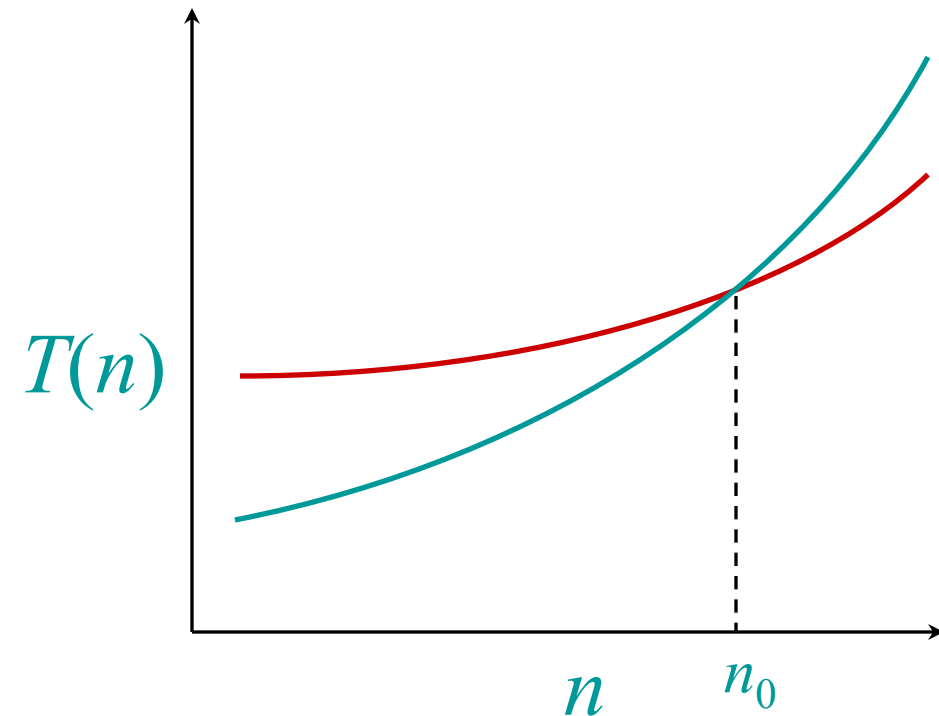


Practical Complexity



Asymptotic Performance

When n gets large enough, a $\Theta(n^2)$ algorithm *always* beats a $\Theta(n^3)$ algorithm.



- Asymptotic analysis is a useful tool to help to structure our thinking toward better algorithm
- We shouldn't ignore asymptotically slower algorithms, however.
- Real-world design situations often call for a careful balancing

Commonly-used Notations

notation	provides	example	shorthand for	used to
Big Theta	asymptotic order of growth	$\Theta(N^2)$	$\frac{1}{2} N^2$ $10 N^2$ $5 N^2 + 22 N \log N + 3N$ $\vdots$	classify algorithms
Big Oh	$\Theta(N^2)$ and smaller	$O(N^2)$	$10 N^2$ $100 N$ $22 N \log N + 3 N$ $\vdots$	develop upper bounds
Big Omega	$\Theta(N^2)$ and larger	$\Omega(N^2)$	$\frac{1}{2} N^2$ N^5 $N^3 + 22 N \log N + 3 N$ $\vdots$	develop lower bounds

Other Asymptotic Notations

- A function $f(n)$ is $o(g(n))$ if $\exists$ positive constants c and n_0 such that

$$f(n) < c g(n) \quad \forall n \geq n_0$$

- A function $f(n)$ is $\omega(g(n))$ if $\exists$ positive constants c and n_0 such that

$$c g(n) < f(n) \quad \forall n \geq n_0$$

- Intuitively,

- | | | |
|------------------------|-----------------------------|--------------------------|
| ▪ $o()$ is like $<$ | ▪ $\omega()$ is like $>$ | ▪ $\Theta()$ is like $=$ |
| ▪ $O()$ is like $\leq$ | ▪ $\Omega()$ is like $\geq$ | |

Insertion sort analysis

Worst case: Input reverse sorted.

$$T(n) = \sum_{j=2}^n \Theta(j) = \Theta(n^2) \quad [\text{arithmetic series}]$$

Average case: All permutations equally likely.

$$T(n) = \sum_{j=2}^n \Theta(j/2) = \Theta(n^2)$$

Is insertion sort a fast sorting algorithm?

- Moderately so, for small n .
- Not at all, for large n .

See You Next Monday!

- Homework 1 due:
Monday, 09/22/2025 by
11:59 PM
- Kean Cup AI
competition!



KEAN UNIVERSITY
1st Kean Cup AI Competition

Go Keanu!
Wow AI is so Cool!

The poster features the Kean University mascot, a blue and white tiger, wearing a blue jersey with "KEAN 90" and light blue shorts. The mascot is surrounded by various AI-related icons such as a brain, a circuit board, a lightbulb, and a gear. The background is a light blue gradient with a faint silhouette of the mascot's head.

Show off what you can do with AI!
Win gift cards + Prizes!

 **KEAN**
Department of
Computer Science
and Technology

Deadline: Sep 14th.
Scan for details. 